

课程笔记

CS224R 课程笔记：Lecture 1 Class Intro

Codex

2026-04-13



视频作者/频道：CS224R / Stanford

发布日期：课程讲次日期：2025-04-02

视频时长：未在本地元数据中提供

视频链接：https://www.youtube.com/watch?v=EvHRQhMX7_w

目录

1 课程定位与本讲学习目标	2
1.1 课程想让学生获得什么能力	2
1.2 课程组织、资源与作业安排	2
1.3 本讲的技术学习目标	2
1.4 本章小结	3
2 为什么要学深度强化学习	3
2.1 超越监督学习：预测会改变未来	3
2.2 应用面广：从机器人到 LLM	4
2.3 从智能视角理解 RL	4
2.4 开放研究问题仍然很多	4
2.5 本章小结	5
3 如何把行为写成数据：状态、观测、动作、轨迹与奖励	5
3.1 顺序决策问题的最小词汇表	5
3.2 状态与观测：Markov 性质为何重要	6
3.3 轨迹与奖励：从单步判断走向长期效果	6
3.4 策略：怎样用神经网络表示行为	7
3.5 本章小结	7
4 强化学习的目标函数与算法版图	7
4.1 长期目标：最大化期望累计奖励	7
4.2 为什么常常需要随机策略	7
4.3 算法类型：课程接下来会走哪几条线	8
4.4 本章小结	8
5 总结与延伸	9

1 课程定位与本讲学习目标

这节课一半是课程导论，一半是真正的技术开篇。讲者先把课程的行政与学习安排讲清楚，再用这段时间建立整门课的共同语言：什么是深度强化学习，为什么它和普通机器学习不一样，课程后续会围绕哪些算法路线展开。

1.1 课程想让学生获得什么能力

从课件和口头说明看，这门课的核心目标不是死记某几个算法，而是形成两层能力：

- 第一层是概念能力：理解深度强化学习到底在解决什么问题，常见对象包括策略、奖励、轨迹、状态与观测。
- 第二层是实现能力：不仅要会读论文，还要能把已有方法真正实现出来，并理解不同方法的使用前提与局限。

因此课程不会试图覆盖强化学习所有理论分支，而是聚焦于最常见、最能落到深度神经网络上的方法族。讲者明确提到 imitation learning、offline / online RL、model-free / model-based RL、multi-task / meta RL，以及 RL for LLMs 和 RL for robots，这相当于给出了整门课的主题地图。

1.2 课程组织、资源与作业安排

本讲的 logistics 不是纯粹行政信息，因为它直接决定后续学习节奏。课程页面、Canvas、Ed、Gradescope 与 office hours 是日常协作基础；作业和项目则对应两条能力培养线：

- 四次两周制作业，占总评一半，重点是用 PyTorch 实作算法并在物理模拟、导航等环境中跑实验。
- 期末项目占另一半，可以做自定义项目，也可以做默认项目。默认项目在这一年新增，主题与用 RL 微调 LLM 有关。

讲者还强调了先修要求：机器学习、基础深度学习、强化学习入门都默认掌握，原因不是要设置门槛，而是这门课会快速进入算法和系统实现，不会停留在最基础的概率与梯度下降复习上。

为什么第一讲仍然要花时间讲课程结构

因为深度强化学习的学习曲线很陡。作业、项目、office hours 和 PyTorch tutorial 这些安排，本质上是在为后面高强度的实现型内容做缓冲。

1.3 本讲的技术学习目标

讲者把本讲目标总结得很具体：

- 什么是 deep reinforcement learning；
- 怎样表示行为，使其可以被优化；
- 怎样把一个真实问题写成强化学习问题。

也就是说，今天并不是讲某个具体算法，而是在搭建最基础的建模框架。只要这一步不牢，后面无论是 imitation learning、policy gradients 还是 actor-critic，都会变成符号堆积。

1.4 本章小结

第一部分的重点是校准预期：CS224R 讲的是可以落到深度网络上的强化学习方法，目标是让学生具备理解与实现现有乃至新兴方法的能力。课程 logistics 在这里并非边角料，而是为了支撑后续的作业节奏与项目实践。

2 为什么要学深度强化学习

2.1 超越监督学习：预测会改变未来

讲者用一个非常直接的对比来切入：监督学习通常是在固定数据集上学习 $f(x) \approx y$ ，而强化学习要学的是行为 $\pi(a | s)$ 。这个差别看似只是符号不同，实质却是问题结构的不同。

How does deep RL differ from other ML topics?

Supervised learning

Given labeled data: $\{(x_i, y_i)\}$, learn $f(x) \approx y$

- directly told what to output
- inputs x are independently, identically distributed (i.i.d.)

Reinforcement learning

Learn behavior $\pi(a | s)$.

- from experience, indirect feedback
- data **not** i.i.d.: actions a affect the future observations.

Behavior can include:



8

图 1: 监督学习与强化学习的根本差别：一个学习静态输入到标签的映射，一个学习会改变未来观测的数据生成行为。

在监督学习里，模型通常被直接告知正确答案，而且样本经常可以被近似看作独立同分布。强化学习里则不同：

- 反馈往往是间接的，不会直接告诉系统“当前应该输出什么动作”；
- 动作会改变下一步看到的状态或观测，因此数据分布会随着策略变化而变化；
- 我们优化的不再只是单步预测精度，而是长期后果。

讲者用一句话概括得很准：AI model predictions have consequences。也正因为预测有后果，才需要一种把“行为”和“长期结果”绑定起来的学习框架。

2.2 应用面广：从机器人到 LLM

讲者随后列出了一组典型场景，说明强化学习并不是只服务于机器人或游戏。机器人控制、聊天系统、游戏智能体、自动驾驶、网页代理都可以看成“顺序决策”问题：系统不断接收信息并做动作，动作又会影响接下来可见的信息。

Why study deep reinforcement learning?

1. Going **beyond supervised** (x, y) examples
 - AI model predictions have consequences! How can we take them into account?
 - When direct supervision isn't available Learn from *any* objective.
2. Widely used and deployed for performant AI systems
3. Learning from experience seems **fundamental to intelligence**
 - RL can **discover new solutions**
4. Plenty of exciting open research problems

17

图 2: 课程把“为什么学习深度强化学习”首先归结为超越静态 (x, y) 学习范式。

这部分还给出了若干更具体的工业与研究实例：腿足机器人、机械臂操作、复杂博弈、交通控制、图像生成模型的对齐，以及现代大语言模型的后训练。其共同点在于，系统不是只要把一个标签猜对，而是要持续地产生高质量行为。

本课程的一个重要立场

强化学习在现代 AI 里并不是边缘课题。尤其当系统需要和环境交互、需要从反馈中持续改进，或者需要围绕复杂目标进行优化时，RL 会自然出现。

2.3 从智能视角理解 RL

讲者给出的第三个理由更加基础：learning from experience seems fundamental to intelligence。一个系统如果只能照着已有标注做预测，而不能在试错中改进，就很难具备真正开放场景中的适应性。

从这个角度看，强化学习之所以重要，不仅因为它在工业上有用，还因为它提供了一个形式化框架，去描述“练习后变好”“从结果中反思”“在不确定环境里试错”这些更接近智能本质的过程。

2.4 开放研究问题仍然很多

第一讲没有回避强化学习的困难。讲者明确指出，即便已经有很多成功应用，深度强化学习依然充满挑战，例如：

- 如何定义或学习合适的 reward；

- 如何让智能体泛化到不同场景；
- 如何在数据昂贵、探索危险、反馈稀疏时稳定学习；
- 如何减少“幕后大量人工支持”的成本。

这也解释了为什么课程会横跨 imitation learning、model-based、RL for LLMs、robotics 等多个主题，因为这些方向分别在解决不同瓶颈。

2.5 本章小结

学习深度强化学习有四个层面的理由：它能处理监督学习难以处理的决策后果问题；它支撑大量现代 AI 系统；它提供了学习于经验的形式化框架；同时它仍然有大量基础与应用研究问题值得解决。

3 如何把行为写成数据：状态、观测、动作、轨迹与奖励

3.1 顺序决策问题的最小词汇表

讲者接着把 RL 的基础记号完整展开。要把一个问题变成强化学习问题，至少要写清这几个对象：

- state s_t ：时刻 t 世界的状态；
- observation o_t ：智能体在时刻 t 实际看到的信息；
- action a_t ：在时刻 t 做出的决策；
- trajectory τ ：状态 / 观测与动作构成的时序序列；
- reward $r(s, a)$ ：状态动作对有多好。

How to represent experience as data?

state $\mathbf{s}_t$ - the state of the “world” at time t

observation $\mathbf{o}_t$ - what the agent observes at time t

(only used when missing information)

action $\mathbf{a}_t$ - the decision taken at time t

trajectory τ - sequence of states/observations and actions

$(\mathbf{s}_1, \mathbf{a}_1, \mathbf{s}_2, \mathbf{a}_2, \dots, \mathbf{s}_T, \mathbf{a}_T)$ (could be length $T=1$!)

reward function $r(\mathbf{s}, \mathbf{a})$ - how good is $\mathbf{s}, \mathbf{a}$?

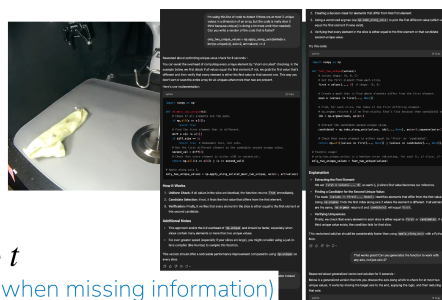


图 3: 把经验表示为数据的基本对象：状态、观测、动作、轨迹与奖励。

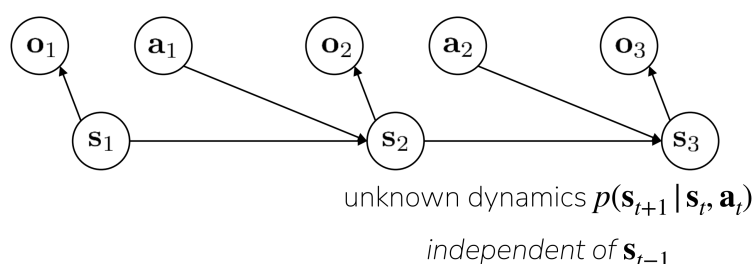
其中最容易混淆的是 state 和 observation。讲者专门提醒，两者并不总相同。状态是“足够描述未来演化”的世界变量；观测则只是智能体拿到的输入，有时它不完整，只是对状态的部分投影。

3.2 状态与观测：Markov 性质为何重要

当下一个状态只依赖当前状态、当前动作以及随机性，而不依赖更久远的历史时，我们说系统满足 Markov property。这个性质之所以重要，是因为它把复杂的历史依赖压缩进当前状态中，让决策问题变得可递推、可估计。

States vs. observations

Next state is purely a function of the current state and action (and randomness)



“Markov property”

33

图 4: 状态、观测与系统动力学的关系：Markov 性质要求未来由当前状态和动作决定。

但在很多现实问题中，智能体拿不到完整状态，只能看到观测。例如对话系统只能看到最近对话历史，机器人可能只能拿到图像与传感器读数。这时如果只把 o_t 喂给策略，信息可能不够，往往要给策略加记忆，让它基于一段观测历史作决策。

3.3 轨迹与奖励：从单步判断走向长期效果

轨迹可以写成

$$\tau = (s_1, a_1, s_2, a_2, \dots, s_T, a_T).$$

符号说明如下：

- T : 轨迹长度，可以固定也可以变化；
- s_t, a_t : 第 t 步的状态与动作；
- 整条轨迹是一次 policy roll-out，也常被叫作一个 episode。

奖励函数则用来把“想要什么行为”写成数值目标。例如：

- 毛巾挂到钩子上给高分；
- 用户给出 upvote 给高分；

- 机器人向前移动得更快给高分。

这一步是强化学习建模最关键也最脆弱的地方。因为奖励写得不好，系统就会学到偏离真实意图但在数值上看起来更优的行为。

3.4 策略：怎样用神经网络表示行为

策略 π_θ 负责把状态或观测映射到动作分布。第一讲里先不给出具体网络结构，而是强调策略是“行为的参数化表示”。这让后续所有算法都可以统一理解为：调整参数 θ ，让策略生成更好的轨迹。

讲者还特别提出 stochastic policies 的动机。引入随机性并不是装饰，而是因为：

- 需要探索时，系统必须尝试不同动作；
- 真实示范数据和现实世界本身都常常是多峰的、随机的；
- 用分布而非单点，更能表示行为不确定性。

3.5 本章小结

把问题写成 RL，需要先定义数据对象，再定义行为对象。状态与观测区分了“世界真实变量”和“智能体实际可见信息”；轨迹与奖励把问题从单步预测升级为长期结果优化；策略则把行为变成可学习的参数化对象。

4 强化学习的目标函数与算法版图

4.1 长期目标：最大化期望累计奖励

讲者把 RL 的核心目标写成：

$$\max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T r(s_t, a_t) \right].$$

这里每个符号都很重要：

- θ ：策略参数；
- $p_{\theta}(\tau)$ ：由当前策略与环境动力学共同诱导的轨迹分布；
- $\sum_{t=1}^T r(s_t, a_t)$ ：一条轨迹上的累计回报；
- $\mathbb{E}$ ：我们优化的是在轨迹分布下的平均表现，而不是单次 rollout 的运气。

这条公式把前面所有概念串了起来。策略不是为了某一步动作像不像专家，而是为了让长期结果更好。

4.2 为什么常常需要随机策略

第一讲给出的随机策略理由很朴素，但非常根本：

- 没有随机性，就很难主动探索没见过的动作；
- 如果现实数据来自多个示范者或多种成功方式，确定性输出容易退化“平均动作”；
- 后续的很多梯度型算法，本来就是基于动作分布来推导的。

因此“策略是分布”并不是技术细节，而是深度强化学习方法设计的基础假设之一。

4.3 算法类型：课程接下来会走哪几条线

在课程结尾，讲者给出一个非常有用的算法地图。不同算法都服务于同一个目标函数，但会在监督来源、数据来源、是否显式估计价值、是否学习动力学模型等方面做不同权衡。

Types of algorithms

$$\text{maximize expected sum of rewards: } \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t^T r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

1. **Imitation learning**: mimic a policy that achieves high reward
2. **Policy gradients**: directly differentiate the above objective
3. **Actor-critic**: estimate value of the current policy and use it to make the policy better
4. **Value-based**: estimate value of the optimal policy
5. **Model-based**: learn to model the dynamics, and use it for planning or policy improvement

41

图 5: 强化学习算法版图：模仿学习、策略梯度、actor-critic、value-based 与 model-based。

这几类方法可以先做粗略理解：

- imitation learning：直接模仿高回报行为；
- policy gradients：直接对上面的期望目标求梯度；
- actor-critic：一边学策略，一边学价值评估来帮助改进策略；
- value-based：先学“什么动作值钱”，再由价值导出策略；
- model-based：显式学习环境动力学，再做规划或利用模型辅助学习。

讲者没有说某条路线绝对更好，而是强调方法选择取决于数据是否便宜、奖励是否清晰、环境是否可控、是否需要高数据效率等条件。

为什么会有这么多 RL 算法

因为强化学习的难点不是单一的。有人缺数据，有人缺稳定性，有人缺可探索性，有人环境危险无法在线试错，所以算法会围绕不同瓶颈做取舍。

4.4 本章小结

强化学习的统一目标是最大化期望累计奖励，但实现这件事可以走多条路线。第一讲的作用，就是先把整个方法空间摆出来，后续每一讲再把其中一类方法拆开讲透。

5 总结与延伸

Lecture 1 完成了两件事。第一，它把 CS224R 这门课的学习框架交代清楚了：课程强调实现、实验和项目，不打算停留在高层概念综述。第二，它用最少但最关键的记号搭建了 RL 的语言体系：状态、观测、动作、轨迹、奖励、策略、期望累计回报。

如果用一句话概括本讲的核心，那么就是：**强化学习研究的是会影响未来数据分布的行为优化问题**。这使它天然不同于静态监督学习，也解释了为什么后续会出现 imitation learning、policy gradient、actor-critic、value-based 和 model-based 这些不同路线。

从学习路径看，第一讲最值得反复消化的不是 logistics，而是三个基本判断：

- 行为学习问题和静态预测问题在数据生成机制上不同；
- 建模前必须先分清 state、observation、reward 和 trajectory；
- 所有后续算法，本质上都在尝试更有效地优化长期回报目标。

后续课程自然会沿着这个框架推进：先从 imitation learning 这样“监督信号更强”的方法出发，再进入真正依赖在线试错的 reinforcement learning 算法。